

DELIVERABLE 3: APPROACH & LOGIC EXPLANATIONS

**Data Vortex Competition — Aaruush '26,
Round 1, Phase 2**

**Selected: E2 (Easy) | M5 (Medium) | H5
(Hard)**

E2 Engineering Logic: Most Engaged Posts

Query Architecture

```

posts table
|
v
[WHERE likes IS NOT NULL] -----> Removes 1,814 posts (15.1%) with
indeterminate engagement
|
v
[Compute total_engagement = likes + shares + comments] -----> Pre-
aggregation in CTE
|
v
[DENSE_RANK() OVER (ORDER BY total_engagement DESC)] -----> Window
function ranking
|
v
[WHERE engagement_rank <= 10] -----> Top-10 filter
|
v
[JOIN users ON user_id] -----> Enrich with user metadata
|
v
RESULT: 10 rows

```

Performance Strategy

1. **CTE Pipeline Architecture:** The `engagement_scored` CTE pre-filters NULLs and computes the aggregate expression **before** the ranking step. This ensures the window function (`DENSE_RANK`) operates on a clean, pre-filtered result set of ~10,186 rows rather than the full 12,000-row table.
2. **CAST(likes AS INTEGER):** The `likes` column is stored as REAL (from the pandas CSV export). We explicitly cast to INTEGER for clean integer display without affecting sort order. This avoids displaying values like `4983.0` instead of `4983`.
3. **DENSE_RANK vs ROW_NUMBER:** We use `DENSE_RANK()` instead of `ROW_NUMBER()` to handle engagement ties fairly. If two posts have identical total engagement scores (e.g., both at 7,500), they share the same rank rather than being arbitrarily ordered. This is semantically correct for a "top 10" leaderboard.

4. **JOIN Placement:** The `JOIN users` is placed in the first CTE, not the final SELECT. This ensures user metadata (location, follower_count) is available throughout the pipeline without requiring a late-stage join on the ranked output.

NULL Treatment

- Posts with `NULL` likes are excluded by the `WHERE p.likes IS NOT NULL` filter in the first CTE
 - This removes 1,814 posts (15.1%) whose engagement scores would be mathematically indeterminate (`NULL + integer = NULL` in SQL)
 - Shares and comments are always non-null in our dataset, so only likes requires this guard
 - The remaining 10,186 posts are ranked on complete data
-

M5 Engineering Logic: Suspicious Engagement Detection

Query Architecture

```

posts table
|
v
[WHERE likes IS NOT NULL] -----> Same NULL guard as E2
|
v
[Filter: shares > (likes + comments)] -----> Core suspicion criterion
|                                           Yields 1,209 posts
(10.1%)
v
[Compute share_surplus = shares - (likes + comments)] -----> Anomaly
magnitude
[Compute share_pct_of_total = shares / total * 100] -----> Ratio
analysis
|
v
[ROW_NUMBER() OVER (ORDER BY shares DESC)] -----> Rank by raw share
volume
[PERCENT_RANK() OVER (ORDER BY share_surplus)] -----> Extremity
percentile
|
v
[WHERE suspicion_rank <= 20] -----> Top-20 filter
|
v
RESULT: 20 rows

```

Performance Strategy

1. **Pre-computation in CTE:** The `suspicious_posts` CTE computes three derived metrics (`likes_plus_comments`, `share_surplus`, `share_pct_of_total`) in a single table scan. The inequality filter `shares > (likes + comments)` is applied within the same CTE, avoiding a separate subquery step.
2. **Ratio Analysis:** `share_pct_of_total` quantifies what percentage of total engagement comes from shares alone. In organic social media engagement:
 - **Normal range:** 20-35% of total engagement from shares

- **Suspicious range:** 50-65% from shares
- **Highly anomalous:** >65% from shares (e.g., post #15 at 81.0%)

3. **Dual Ranking Strategy:** We use two different window functions:

- `ROW_NUMBER() OVER (ORDER BY shares DESC)` — ranks by raw share volume (the sort criterion requested)
- `PERCENT_RANK() OVER (ORDER BY share_surplus ASC)` — shows how extreme each case is within the suspicious set

4. **NULLIF Guard:** `NULLIF(..., 0)` in the denominator of `share_pct_of_total` prevents division-by-zero errors for posts where all metrics might sum to zero.

Condition Handling

- The condition `shares > (CAST(p.likes AS INTEGER) + p.comments)` directly encodes the problem statement
- Only posts with non-null likes are evaluated to prevent false positives from NULL arithmetic (`NULL > X` always evaluates to `NULL/FALSE`, but we exclude them explicitly for clarity)
- The `JOIN users` enriches results with location data that reveals geographic clustering patterns (Mexico City appears 3x in top 20)

H5 Engineering Logic: Data Anomaly Identification

Query Architecture

```

posts_raw table (12,360 records)
|
|---> [Category 1: CAST(likes AS REAL) < 0]          ----> 525
records
|
|---> [Category 2: platform IS NULL OR '' OR 'NULL']  ----> 1,846
records
|
|---> [Category 3: text IS NULL OR '' OR 'NULL']      ----> 1,746
records
|
|---> [Category 4: LIKE '%<br>%' OR '%&%' etc.]      ----> 1,004
records
|
v
[UNION ALL] ----> Combine all anomalies (5,121 total)
|
|               NOTE: UNION ALL, not UNION, preserves multi-
category records
v
[COUNT(*) OVER (PARTITION BY anomaly_type)] ----> Per-category totals
[ROW_NUMBER() OVER (PARTITION BY anomaly_type ORDER BY post_id)]
|
v
[WHERE row_within_category <= 5] ----> Top 5 examples per category
|
v
RESULT: 20 rows (5 x 4 categories)

```

Performance Strategy

1. **UNION ALL (not UNION)**: Uses `UNION ALL` because we explicitly want to count a record in multiple categories if it has multiple anomaly types. For example, `post_id = 01kgwhi645er` appears in both `NEGATIVE_LIKES` and `MISSING_PLATFORM` categories. Using `UNION` would deduplicate across categories and undercount the true anomaly exposure.

2. **Runs Against RAW Data:** The query targets `posts_raw` (the original corrupted CSV), NOT the cleaned table. This is critical because Phase 1 cleaning already:

- Resolved negative values (absolute value)
- Stripped HTML tags and entities
- Converted 'NULL' strings to proper NaN

Running H5 against cleaned data would return zero results for `NEGATIVE_LIKES` and `HTML_CONTAMINATION` categories.

3. **Window Function for Counting:** `COUNT(*) OVER (PARTITION BY anomaly_type)` provides the category total alongside each sample row in a single pass, avoiding a separate aggregation query or self-join.

4. **Supplementary Aggregation Query:** A second query provides the roll-up summary (total counts, affected users, percentage of total) for the insight report. This uses a subquery with the same `UNION ALL` pattern, then applies `GROUP BY` for aggregation.

String Filtering Logic for HTML Detection (Category 4)

The HTML contamination scan uses six `LIKE` patterns in an `OR` chain:

```
-- HTML Tags (structural injection)
WHERE text_content LIKE '%<br>%'           -- Line break tag
      OR text_content LIKE '%<div>%'       -- Division open tag
      OR text_content LIKE '%</div>%'      -- Division close tag

-- HTML Entities (encoding failure)
      OR text_content LIKE '%&amp;%'      -- Ampersand entity
      OR text_content LIKE '%&lt;%'       -- Less-than entity
      OR text_content LIKE '%&gt;%'       -- Greater-than entity
```

- **Tags** (`<br>` , `<div>` , `</div>`) indicate HTML-to-plaintext conversion failure in the export pipeline. The raw HTML structure was not stripped before CSV serialization.
- **Entities** (`&` , `<` , `>`) indicate double-encoding: the text was HTML-entity-encoded but never decoded back to plain characters.
- The `CASE` expression in `flagged_value` identifies which specific artifact was found first, providing diagnostic precision for root-cause analysis.

NULL Treatment (Three-Way Check for Category 2 & 3)

```
WHERE text_content IS NULL          -- True SQL NULL
      OR TRIM(text_content) = ''    -- Empty string (zero-
length)
      OR TRIM(UPPER(text_content)) = 'NULL' -- Literal 'NULL' sentinel
string
```

This three-pronged check catches all forms of "missing" data:

1. **True NULL:** Standard SQL null from pandas `.to_sql()` export
2. **Empty string:** From malformed CSV rows where the field was present but empty
3. **'NULL' string:** The corrupted export pipeline used the literal string `"NULL"` as a sentinel value instead of proper null markers (detected 24 times in `text_content` during Phase 1 profiling)

The `TRIM()` and `UPPER()` wrappers handle whitespace-padded or case-variant 'null' strings.